컴퓨터 시스템 프로그래밍 — 9/03 강의 퀴즈

- **출제 범위** — 02-machine-basics (슬라이드 5~34), 03-machine-control (슬라이드 1~14)
- **구성** — OX 5문항 · 서술형 10문항 · 예제 5문항 (총 20문항)
- **이름** — _____

Part 1. OX 문제 (5문항)

1. `leaq` 는 오퍼랜드에 `(%rsi,%rsi,2)` 같은 괄호 표기를 사용하지만, 메모리를 읽지 않는다. (O / X)

2. CPU 캐시의 크기와 상태는 ISA(아키텍처)의 일부이므로, 어셈블리 프로그래머가 반드시 알고 있어야 한다. (O / X)

3. x86-64에서는 `movq` 명령 하나로 메모리에 있는 값을 다른 메모리 위치로 직접 옮길 수 있다. (O / X)

4. `cmpq %rsi, %rdi` 는 뺄셈 결과를 데스티네이션 레지스터에 저장한 뒤 컨디션 코드를 설정한다. (O / X)

5. 64비트 x86-64 CPU는 하위 호환성(backward compatibility) 때문에 32비트용으로 컴파일된 머신 코드도 실행할 수 있다. (O / X)

Part 2. 서술형 문제 (10문항)

각 문항은 2~4문장으로 답하시오.

6. 아키텍처(ISA)와 마이크로아키텍처의 차이를 설명하고, 각각에 해당하는 예를 하나씩 드시오.

7. “Programmer-Visible State”에 해당하는 것 세 가지를 쓰고, CPU 캐시가 여기에 포함되지 않는 이유를 설명하시오.

8. 로드(load)와 스토어(store)를 각각 정의하고, 강의에서 제시한 “헷갈리지 않는 기준”이 무엇인지 쓰시오.

9. 어셈블리 코드와 머신 코드의 대응 관계, 그리고 C 코드와 어셈블리 코드의 대응 관계가 어떻게 다른지 설명하시오.

10. `movb $0x05, %al` 에서 `b` , `$` , `%` 가 각각 무엇을 의미하는지 쓰시오.

11. `movq` 의 오퍼랜드 조합 표에서, 이미디어트(immediate)가 데스티네이션 자리에 올 수 없는 이유를 설명하시오.

---

12. gcc가 C 소스 파일에서 실행 파일을 만들기까지 거치는 세 단계를 순서대로 쓰고, 각 단계의 입력과 출력을 쓰시오.

---

13. `sarq` 와 `shrq` 의 동작 차이를 설명하고, signed 정수를 오른쪽으로 시프트할 때 어느 것을 써야 하는지 쓰시오.

---

14. 컨디션 코드의 implicit setting과 explicit setting의 차이를 설명하고, explicit setting에 해당하는 명령어를 두 개 쓰시오.

---

15. 완전한 어드레싱 모드 `D(Rb,Ri,S)` 가 x86에 존재하게 된 역사적 이유를 강의 내용에 근거해 설명하시오.

Part 3. 예제 문제 (5문항)

16. 다음 코드가 실행된 직후 CF, ZF, SF, OF 각각의 값(0 또는 1)을 쓰시오.

```
    movq  $5, %rax
    subq  $5, %rax
```

CF	ZF	SF	OF

17. 다음 두 줄이 하는 일을 C의 `if` 문 한 줄로 표현하시오. (`%rax`에 변수 `x`가 들어 있다고 가정)

```
    testq  %rax, %rax
    je     .L1
```

---

18. `%rdx = 0xF000`, `%rcx = 0x0100` 일 때 다음 어드레스 표현이 가리키는 주소를 계산하시오.

a. `(%rdx,%rcx,4)` = _____

b. `0x80(,%rdx,2)` = _____

---

19. 다음 함수가 반환하는 값을 인자 `x`에 대한 식으로 쓰고, 컴파일러가 `imulq`를 쓰지 않은 이유를 한 문장으로 설명하시오.

```
foo:
    leaq  (%rdi,%rdi,4), %rax
    salq  $2, %rax
    ret
```

---

20. 다음 함수를 C로 복원하시오. 또한 `movq`를 `leaq`로 바꾸면 반환값이 어떻게 달라지는지 쓰시오.

```
bar:
    movq  (%rdi,%rsi,8), %rax
    ret
```

정답

Part 1. OX

1. O — `lea`는 Load Effective Address. 괄호 안의 주소 식을 계산만 하고 역참조하지 않는다. 메모리 접근 0회.

2. X — 캐시 크기와 상태는 마이크로아키텍처다. 어셈블리 프로그래머가 몰라도 CPU가 알아서 처리한다.

3. X — 메모리-투-메모리 전송 명령은 존재하지 않는다. 소스와 데스티네이션 중 하나만 메모리 오퍼랜드가 될 수 있다.

4. X — `cmpq`는 `a - b`를 계산하되 **결과를 저장하지 않고** 컨디션 코드만 설정한다.

5. O — 옛 레지스터(`%al`, `%eax`)를 버리지 않고 유지하면서 확장만 해왔다. 레지스터 이름이 지저분한 이유이기도 하다.

Part 2. 서술형

6. 아키텍처(ISA)는 CPU가 외부에 제공하는 추상화, 즉 "CPU를 쓰는 사람이 알아야 하는 명세"다. 인스트럭션 집합과 레지스터가 그 예다. 마이크로아키텍처는 그 아키텍처를 실제로 구현한 내부 디테일로, 캐시 크기·코어 주파수·파이프라인 단계가 예다. C++에 비유하면 아키텍처는 클래스의 인터페이스, 마이크로아키텍처는 멤버 함수의 구현이다.

7. PC(프로그램 카운터, x86-64에서는 `%rip`), 레지스터 파일, 컨디션 코드. 캐시는 프로그램의 **결과**를 바꾸지 않고 성능에만 영향을 주며, CPU가 알아서 관리하므로 어셈블리 프로그래머가 볼 이유가 없다. 즉 마이크로아키텍처에 속한다.

8. 로드는 메모리에서 레지스터로 데이터를 가져오는 것, 스토어는 레지스터의 값을 메모리로 내려보내는 것이다. 기준은 **항상 CPU를 주어로 놓고 생각하는 것**이다. 메모리는 수동적(passive) 디바이스이고 CPU가 능동적인 주체이므로, "CPU가 가져온다 = 로드, CPU가 저장한다 = 스토어"로 읽으면 헷갈리지 않는다.

9. 어셈블리 코드와 머신 코드는 사실상 1대1 대응이다. 같은 것을 사람이 읽을 수 있는 형태로 쓰느냐, 16진수 인코딩으로 쓰느냐의 차이일 뿐이다. 반면 C 코드와 어셈블리 코드는 1대1이 아니라 N대N 관계다. C 한 줄이 어셈블리 여러 줄이 될 수 있고, 컴파일러나 최적화 옵션에 따라 사용하는 레지스터와 명령어가 달라진다.

10. `b`는 오퍼레이션 서픽스로, 옮기는 데이터의 크기가 1바이트(byte)임을 뜻한다. `$`는 뒤에 오는 것이 이미디어트, 즉 상수 값임을 나타내는 접두사다. `%`는 뒤에 오는 것이 레지스터 이름임을 나타내는 접두사다.

11. 이미디어트는 명령어 안에 인코딩된 상수 값이므로 저장 공간이 아니다. 데스티네이션은 연산 결과가 쓰여야 하는 곳인데, 상수에는 값을 쓸 수 없다. 그래서 표에서 이미디어트가 데스티네이션인 조합은 애초에 존재하지 않는다.

12. (1) 컴파일러: C 프로그램(`.c`)을 입력받아 어셈블리 프로그램(`.s`)을 출력한다. (2) 어셈블러: 어셈블리(`.s`)를 입력받아 오브젝트 프로그램(`.o`, 머신 코드)을 출력한다. (3) 링커: 오브젝트 파일들과 라이브러리를 입력받아 실행 파일을 출력한다. `gcc` 커맨드 한 줄은 이 세 단계를 내부에서 모두 호출한 것이다.

13. `sarq`는 arithmetic right shift로, 빈 최상위 비트를 **사인 비트로 채운다**. 따라서 원래 음수였으면 시프트 후에도 음수를 유지한다. `shrq`는 logical right shift로 최상위 비트를 무조건 0으로 채운다. signed 정수를 시프트할 때는 부호가 보존되는 `sarq`를 써야 한다.

14. implicit setting은 프로그래머가 플래그를 직접 쓰지 않았는데도 `addq`, `subq` 같은 산술·논리 연산의 **부수 효과(side effect)**로 컨디션 코드가 설정되는 것이다. explicit setting은 플래그를 세팅하는 것 자체가 목적인 명령어를 쓰는 경우로, `cmpq`와 `testq`가 여기 해당한다.

15. 컴파일러가 없던 시절에는 프로그래머가 어셈블리를 직접 손으로 짰다. 그때 가장 흔한 작업이 배열을 카운터 변수로 순회하는 루프였고, `베이스 주소 + 원소크기 × 인덱스 + 옵셋`이라는 계산이 매번 필요했다. 이 계산을 한 줄로 처리할 수 있게 어드레싱 모

드를 만들어둔 것이며, 없었다면 매번 서너 줄을 더 써야 했다. 지금은 컴파일러가 짜주지만 이 모드는 그대로 남아 사용된다.

Part 3. 예제

16.

CF	ZF	SF	OF
0	**1**	0	0

`5 - 5 = 0` 이므로 결과가 0 → ZF = 1. 결과가 음수가 아니므로 SF = 0. 뺄셈에서 borrow가 발생하지 않았으므로 CF = 0. signed 오버플로도 없으므로 OF = 0.

17.

```
        if (x == 0) goto .L1;
```

`testq %rax, %rax` 는 `%rax & %rax` 를 계산하는데 그 결과는 `%rax` 자신이다. `%rax` 가 0이면 ZF가 셋 되고, `je` (jump if equal)는 ZF가 셋일 때 점프하므로 결국 "`x` 가 0이면 점프"가 된다. `%rax` 를 파괴하지 않고 0 여부만 검사할 수 있어 가장 흔히 쓰이는 패턴이다.

18.

a. `(%rdx,%rcx,4)` = `0xF000 + 4 × 0x100` = `0xF000 + 0x400` = **0xF400**

b. `0x80(,%rdx,2)` = `2 × 0xF000 + 0x80` = `0x1E000 + 0x80` = **0x1E080**

(b)는 베이스 레지스터 자리가 비어 있고 인덱스만 있는 형태임에 주의.

19.

반환값은 **`20 × x`** 이다.

- `leaq (%rdi,%rdi,4), %rax` → `%rax = x + 4x = 5x`
- `salq $2, %rax` → `%rax = 5x << 2 = 20x`

`imulq` 는 레이턴시가 큰 명령이라, 곱하는 수가 상수일 때는 `leaq` 와 시프트의 조합으로 더 빠르게 계산할 수 있기 때문이다. (20 = 5 × 4)

20.

```
        long bar(long *p, long i) {
            return p[i];
        }
```

`%rdi` 가 첫 번째 인자 `p` (배열의 시작 주소), `%rsi` 가 두 번째 인자 `i` 이며, 스케일 8은 `long` 의 크기다. 따라서 `Mem[p + 8i]` , 즉 `p[i]` 의 **값**을 반환한다.

`movq` 를 `leaq` 로 바꾸면 메모리를 읽지 않고 주소 계산만 하므로, 반환값이 `p[i]` 의 값이 아니라 **`&p[i]` , 즉 그 원소의 주소**가 된다.